

# Contemporary C++: *Learning Modern C++ in a Modern Way*



مدرس: سعید امراللہی بیوکی

Saeed Amrollahi Boyouki

[www.Linkedin.com/in/saeed-amrollahi-boyouki](http://www.Linkedin.com/in/saeed-amrollahi-boyouki)

# Agenda 10/30

## Introduction to Functions: Part 1

- Functions
- Writing simple functions
- Argument passing: copy semantics vs. Reference semantics
- References and further readings
- Q & A

120 min (incl. Q & A)



# Functions

- Program organization: Data structures, Functions, User-defined types
- Function is one of fundamental tools for *abstraction* in C++.
- A **function** is a piece of program that has a name, and that another part of the program can *call*, or cause to run.

- function declaration:

- function name
- return type
- number and type of arguments

```
double sqrt(double); // just declaration
double sqrt(double x) { // decl. and def.
    // ... compute the square root of x and return it;
}
void exit(int exit_code); // declaration
```

- function definition:

- function declaration
- function body

Return type

Function name

Argument type  
and name

```
long int sqr(int x)
{
    return x * x;
}
```

Function body

# Function call

- Use `sqr` and `sqrt`.

```
double sr2 = sqrt(2); // call sqrt with argument double(2)
long s2 = sqr(2); // call sqr with argument 2
double sr3 = sqrt("three"); // error: sqrt() requires an argument of type double
array<int, 4> a = { 0, 1, 2, 3 };
long s4 = sqr(a); // error: sqr() requires an argument of type int
```

- The semantics of *argument passing* are identical to the semantics of *initialization*.
- The type of the definition and all declarations for a function must specify the same type.

Actual  
argument

```
long int sqr(int x)
{
    return x * x;
}

int main()
{
    long X = sqr(10); // function call
}
```

Formal argument

# Functions: some examples

```
/* compute a student's overall grade from  
   midterm and final exam grades and homework grade */  
double grade(double midterm, double final, double homework)  
{  
    return 0.2 * midterm + 0.4 * final + 0.4 * homework;  
}
```

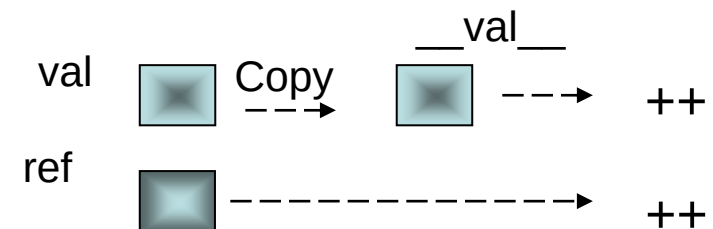
```
// return the maximum value of two integers  
int Max(int x, int y)  
{  
    return x > y ? x : y;  
}
```

  
*Print Square Function*  
Prog.

# Argument passing, pass by value, pass by reference

- Formal arguments vs. Actual arguments
- Initialization
- Type checking and type conversion
- Pass by value vs. Pass by reference

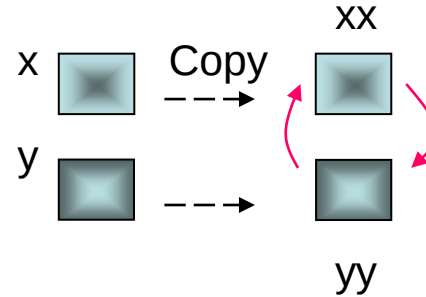
```
void f(int val, int& ref)
{
    val++;
    ref++;
}
void g()
{
    int i = 1;
    int j = 1;
    f(i,j); // i == 1, j == 2
}
```



# Pass by value vs. Pass by reference: swap example

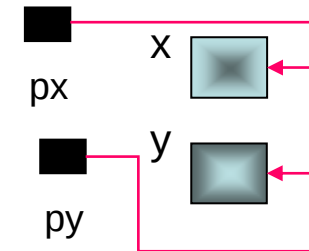
- Pass-by value

```
/* wrong */  
void swap(int x, int y)  
{  
    int tmp;  
    tmp = x;  
    x = y;  
    y = tmp;  
}
```



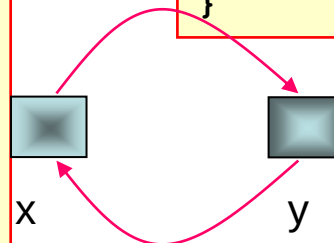
- Pass-by reference: C solution

```
/* right */  
void swap(int* px, int* py)  
{  
    int tmp;  
    tmp = *px;  
    *px = *py;  
    *py = tmp;  
}
```



- Pass-by reference: C++ solution

```
/* right */  
void swap(int& x, int& y)  
{  
    int tmp;  
    tmp = x;  
    x = y;  
    y = tmp;  
}
```



# R references and further readings

[KR78] Brian W. Kernighan & Dennis M. Ritchie. The C Programming Language, Prentice-Hall, Englewood Cliffs, NJ, 1978.

[Str00] Bjarne Stroustrup. The C++ Programming Language (Special Edition). Addison Wesley. Reading Mass. USA. February 2000.



*Thanks for your patience ...*

A man who asks a question is a fool for minute,  
The man who does not ask, is a fool for a life.  
- Confucius

Learning to ask the right (often hard) questions is an essential part of learning to think as a programmer.

- Bjarne Stroustrup *programming Principles and Practice Using C++, page 4.*

There is no stupid question, but there is stupid answer.  
- Howard Hinnant

